

PRODUCT REQUIREMENTS DOCUMENT

AutoSecDev

Multi-Agent AI Security Pipeline for DevSecOps

Prepared for: **Antigravity** | Version: **1.0** | Date: **April 2026**

Project Name	AutoSecDev — Multi-Agent DevSecOps Security Pipeline
Client	Generative AI Course Group (3 members)
Prepared For	Antigravity Development Team
Document Version	1.0 — Initial Release
Date	April 28, 2026
Deadline (Task 1)	April 30, 2026 — 50% working implementation
Deadline (Task 2)	~May 10, 2026 — Full project + research paper
Status	ACTIVE — In Development

 Purpose of this Document

This PRD defines the full scope, architecture, agent specifications, datasets, technical stack, and delivery milestones for the AutoSecDev project. It is intended to give the Antigravity team a complete, unambiguous specification to begin implementation immediately.

1 EXECUTIVE SUMMARY

AutoSecDev is a multi-agent AI system that automatically detects security vulnerabilities in Python code, generates patches, and reports results — all triggered by a GitHub pull request. The system integrates into existing CI/CD pipelines via a GitHub webhook, making it a production-style DevSecOps tool.

The project serves two purposes: (1) a real-world, portfolio-grade engineering project for three final-semester Generative AI students, and (2) a research paper submitted in Springer LNCS format comparing fine-tuned baseline models against the full multi-agent pipeline.

 Core Research Hypothesis

A multi-agent system combining LLM reasoning with traditional SAST tools and RAG-grounded patch generation outperforms standalone fine-tuned models (CodeBERT / CodeT5) in both vulnerability detection accuracy and patch correctness rate.

2 OBJECTIVES

2.1 Engineering Objectives

- Build a 3-agent pipeline: SAST Agent → Patch Agent → Report Agent
- Integrate pipeline with GitHub via webhooks so it triggers on every pull request
- Post results as automated PR comments with vulnerability findings and suggested patches
- Achieve stable end-to-end operation on Python codebases
- Build a Streamlit UI as a manual demo interface (backup for presentation)

2.2 Research Objectives

- Fine-tune CodeBERT on CVEfixes dataset as detection baseline (Model 1)
- Fine-tune CodeT5 on CVEfixes before/after pairs as patch baseline (Model 2)
- Evaluate both baselines vs. multi-agent system on SecurityEval benchmark
- Produce quantitative results: F1, precision, recall, patch correctness rate
- Write full research paper in Springer LNCS format with 8–10 cited papers

2.3 Career / Portfolio Objectives

- Demonstrate agentic AI, RAG pipelines, and CI/CD integration on resume
- Produce a citable, publishable research paper as supplementary credential

3 SCOPE

✓ In Scope	✗ Out of Scope
SAST Agent (Bandit + Semgrep + LLM reasoning)	Code Generation Agent (dropped — users paste code)
RAG-grounded Patch Agent with self-verification loop	PDF report generation (HTML/markdown output only)
Report Agent (structured text / HTML output)	Multi-language support (Python only for v1.0)
GitHub webhook + PR comment integration	Ops monitoring agent / live log analysis
CodeBERT fine-tune baseline (Model 1)	Deployment to cloud / production hosting
CodeT5 fine-tune baseline (Model 2)	Frontend beyond basic Streamlit UI
Streamlit UI for manual demo	Support for private/proprietary codebases

4 SYSTEM ARCHITECTURE

AutoSecDev is composed of three primary agents, two fine-tuned baseline models, a RAG knowledge base, and a CI/CD integration layer. The flow is linear with a feedback loop between the SAST and Patch agents.

Pipeline Flow

GitHub Push → Webhook → FastAPI Endpoint → [SAST Agent → Patch Agent (+ RAG) → SAST Re-scan loop] → Report Agent → PR Comment Posted

Layer	Component	Technology	Output
Dev	GitHub Repository	GitHub Webhooks + FastAPI	Webhook POST to pipeline
Sec — Detection	SAST Agent	Bandit + Semgrep + LLM (Claude/GPT-4o)	Vulnerability JSON report
Sec — Patching	Patch Agent + RAG	ChromaDB + CodeT5/Claude + difflib	Patched code + diff
Sec — Verify	Self-verification loop	SAST Agent re-run (max 3 iterations)	Verified or escalated
Sec — Report	Report Agent	Jinja2 template + GitHub API	PR comment with findings
Ops	CI/CD Integration	GitHub Actions / webhook trigger	Automated pipeline execution
UI	Streamlit Demo	Streamlit + Python	Manual paste-and-scan interface

5 AGENT SPECIFICATIONS

5.1 SAST Agent

The SAST Agent is the entry point of the security pipeline. It operates in two stages.

- Stage 1 (Tool Layer): Runs Bandit and Semgrep on the submitted Python code. Both tools are called as Python subprocesses and return structured JSON output. Bandit covers 35+ Python-specific security issues. Semgrep applies 3,000+ open-source security rules.
- Stage 2 (LLM Reasoning Layer): The combined Bandit + Semgrep JSON output and original code are passed to an LLM with a prompt that instructs it to: (a) confirm or dismiss each finding as a true/false positive based on code context, (b) assign severity level, (c) identify CWE ID, and (d) output a clean structured vulnerability report.
- Output schema: { file, line, cwe_id, severity, description, code_snippet, confirmed: true/false }
- False positive filtering is a key research contribution — the LLM reasoning layer reduces noise from rule-based tools.

5.2 Patch Agent

The Patch Agent receives the confirmed vulnerability report and generates a secure fix using RAG-grounded prompting.

- RAG Step: Queries ChromaDB with the CWE ID and vulnerability description. Retrieves top-3 matching CWE explanations, secure coding guidelines, and example vulnerable → fixed code pairs from the CVEfixes knowledge base.
- Prompt Construction: Combines original vulnerable code + vulnerability metadata + RAG-retrieved context into a structured prompt. LLM is instructed to fix only the vulnerable section, not rewrite the entire function.
- Output: Patched code + line-level diff (generated via Python difflib) + natural language explanation of what was changed and why.
- Self-Verification Loop: After patch generation, the patched code is re-passed to the SAST Agent. If the same vulnerability is still detected, the Patch Agent retries with a modified prompt. Maximum 3 iterations before escalating to the Report Agent as unresolved.

5.3 Report Agent

The Report Agent collects outputs from the SAST and Patch Agents and formats them for delivery.

- Formats a structured security report including: vulnerability summary, severity breakdown, before/after code diffs, patch explanations, and unresolved findings.
- Posts the report as a comment on the triggering GitHub Pull Request via the GitHub REST API.
- Also renders the report in the Streamlit UI for manual demo sessions.
- Output format: Markdown (for PR comment) + rendered HTML (for Streamlit).

6 FINE-TUNED BASELINE MODELS

Why Baselines?

The research paper requires a comparison. Fine-tuned standalone models serve as the baseline. The multi-agent system is the proposed improvement. The paper measures whether the full pipeline outperforms the baselines on detection accuracy and patch quality.

6.1 Model 1 — Vulnerability Detector (CodeBERT)

Attribute	Detail
Base Model	microsoft/codebert-base (HuggingFace)
Task	Binary classification: vulnerable (1) / not vulnerable (0)
Training Data	CVEfixes v1.0.8 — Python functions only (~3,000–5,000 samples)
Fine-tune Method	LoRA (Low-Rank Adaptation) via HuggingFace PEFT library — runs on Colab free tier
Evaluation Metrics	F1 score, Precision, Recall, ROC-AUC on SecurityEval benchmark
Estimated Time	2–3 hours fine-tuning on Google Colab T4 GPU

6.2 Model 2 — Patch Generator (CodeT5)

Attribute	Detail
Base Model	Salesforce/codet5-base (HuggingFace)
Task	Sequence-to-sequence: vulnerable code → patched code
Training Data	CVEfixes v1.0.8 — (before_fix, after_fix) Python pairs (~2,000–3,000 pairs)
Fine-tune Method	LoRA via HuggingFace PEFT — seq2seq training objective
Evaluation Metrics	CodeBLEU score, patch correctness rate (does patched code pass SAST re-scan?)
Estimated Time	3–4 hours fine-tuning on Google Colab T4 GPU

7 DATASETS

Dataset	Used For	Size (Python subset)	Access	Priority
CVEfixes v1.0.8	Train Model 1 + Model 2 + RAG KB	~3,000–5,000 functions	Zenodo (free)	PRIMARY
DiverseVul	Augment Model 1 training	~7,500 functions	GitHub (free)	HIGH
BigVul	Supplementary Model 1	Large (noisy — 25% accuracy)	GitHub (free)	MEDIUM
SecurityEval	Evaluate full system	130 samples / 75 vuln types	GitHub (free)	EVAL ONLY
Juliet Test Suite	Evaluation supplementary	Thousands of CWE samples	NIST (free)	EVAL ONLY
MITRE CWE JSON	RAG knowledge base	272 CWE definitions	MITRE (free)	RAG ONLY

⚠ Dataset Note on BigVul

Research has found BigVul label accuracy is only ~25%. Use it only to increase training volume for Model 1, but report this limitation in the paper. Rely on CVEfixes as the primary clean dataset. Acknowledge label noise — this shows research maturity.

8 TECHNICAL STACK

Category	Technology	Purpose
LLM / API	Claude API (primary) or GPT-4o / CodeLlama-7B via Ollama (free fallback)	SAST reasoning + Patch generation
SAST Tools	Bandit, Semgrep (open-source)	Rule-based vulnerability detection
Agent Framework	LangChain or AutoGen	Agent orchestration + tool calling
RAG / Embeddings	sentence-transformers/all-MiniLM-L6-v2 + ChromaDB	CWE knowledge base retrieval
Fine-tuning	HuggingFace Transformers + PEFT (LoRA)	Baseline model training
CI/CD Integration	FastAPI + GitHub Webhooks + GitHub REST API	Trigger pipeline + post PR comments
UI	Streamlit	Manual demo interface
Diff / Code	Python difflib, subprocess, json	Code diffs + tool subprocess calls

Training Runtime	Google Colab (T4 GPU, free tier)	Fine-tuning baseline models
Paper	Overleaf (LaTeX) + Springer LNCS template	Research paper compilation

9 TEAM & RESPONSIBILITIES

The project team consists of 3 students. Responsibilities are divided by agent/layer to enable parallel development. All three meet daily for 15-minute syncs to align on interfaces.

Person	Role	Responsibilities	Deliverable
A	SAST & Detection	SAST Agent (Bandit + Semgrep + LLM layer), fine-tune CodeBERT baseline (Model 1), evaluation on SecurityEval	SAST Agent module + Model 1 weights + eval results
B	Patch & RAG	Patch Agent, ChromaDB RAG setup, self-verification loop, fine-tune CodeT5 baseline (Model 2), Report Agent	Patch Agent module + RAG KB + Model 2 weights + Report Agent
C	Integration & Ops	FastAPI endpoint, GitHub webhook setup, PR comment posting, Streamlit UI, pipeline glue code, paper writing coordination	Working CI/CD pipeline + Streamlit UI + integrated end-to-end system

 **Critical Alignment Rule**

All three agents communicate via a shared Python dictionary schema agreed on Day 1. Person A's SAST Agent output must exactly match what Person B's Patch Agent expects as input. Define and freeze the schema before anyone writes agent code. This single decision prevents the most common integration failure.

10 2-WEEK DELIVERY PLAN

Week 1 — Build Individual Components

Day	Person A	Person B	Person C
1–2	Set up repo, install Bandit + Semgrep, test on 5 sample vulnerable Python files, define shared schema	Set up ChromaDB, download MITRE CWE JSON, build embedding pipeline, test retrieval	Set up FastAPI skeleton, configure GitHub webhook on test repo, verify POST events arriving
3–4	Build SAST Agent: subprocess wrapper + LLM reasoning layer + JSON output. Start CodeBERT fine-tune on Colab.	Build Patch Agent: RAG retrieval + LLM patch prompt + diff lib diff. Start CodeT5 fine-tune on Colab.	Connect webhook → FastAPI → stub pipeline call. Test GitHub PR comment posting via API.
5	Test SAST Agent on 10 real	Build self-verification loop	Build basic Streamlit UI

	CVEfixes Python samples. Record false positive rate.	(SAST re-scan after patch). Test on same 10 samples.	(paste code → run pipeline → show results). Connect all agent modules.
--	---	---	--

Week 2 — Integration, Evaluation & Paper

Day	All Team
6–7	Full integration testing. Run complete pipeline on 20–30 vulnerable samples. Fix interface bugs between agents. Confirm GitHub webhook → PR comment flow works end-to-end.
8	Run formal evaluation: all three models (Model 1, Model 2, Multi-Agent system) on SecurityEval benchmark. Record all metrics. Generate comparison tables.
9–10	Write research paper in Overleaf (LNCS format). A writes Intro + Related Work, B writes System Design + Methodology, C writes Results + Conclusion. Combine and review.
11–12	Code cleanup, comments, README documentation. Paper final review and revision. Prepare demo script for Task 1 submission.
13–14	BUFFER — bug fixes, paper polish, submission prep. Do not schedule feature work here. This buffer is mandatory, not optional.

11 RESEARCH DESIGN & EVALUATION

11.1 Experimental Setup

All three models are evaluated on the same test set: the SecurityEval benchmark (130 samples across 75 vulnerability types). This enables direct comparison.

Baseline 1: CodeBERT	Baseline 2: CodeT5	Full System: AutoSecDev
Fine-tuned on CVEfixes (Python)	Fine-tuned on CVEfixes before/after pairs	SAST Agent + RAG Patch Agent + self-verification
Metrics: F1, Precision, Recall, AUC	Metrics: CodeBLEU, patch correctness rate	Metrics: F1, Precision, Recall, patch rate, false positive reduction

11.2 Expected Research Contributions

- 1. Comparison of fine-tuned models vs. multi-agent LLM pipeline for vulnerability detection
- 2. Demonstration that RAG-grounded patching improves patch correctness over standalone seq2seq generation
- 3. Self-verification loop as a design pattern for iterative vulnerability remediation
- 4. CI/CD integration as the Ops layer completing the DevSecOps cycle

11.3 Papers to Cite (bibliography)

#	Paper	Venue	Cite For
---	-------	-------	----------

1	LLMs in Software Security: A Survey of Vulnerability Detection Techniques	ACM Computing Surveys 2025 (Q1)	Related work
2	LLMDFA: Analyzing Dataflow in Code with LLM	NeurIPS 2024	SAST grounding
3	APPATCH: Adaptive Prompting LLMs for Vulnerability Patching	USENIX Security 2025	Patch design
4	LLM for Automated Vulnerability Repair: Patch Validation Feedback	ACM Alware 2024	Verify loop
5	Fix Pattern-Aware Vulnerability Patch Generation via ICL	ACM TOSEM 2026 (Q1)	RAG patching
6	AutoSafeCoder: Multi-Agent Framework for Secure Code Generation	arXiv 2024	Closest prior work
7	LLM-Based Multi-Agent Systems for Software Engineering	ACM TOSEM 2025 (Q1)	Multi-agent arch
8	Systematic Study of LLM Architectures for Automated Patching	arXiv 2026	Pipeline design
9	RESCUE: Retrieval Augmented Secure Code Generation	ACM CCS 2024	RAG pipeline
10	AI-Powered Vulnerability Detection and Patch Management: Systematic Review	MDPI MAKE 2026	Introduction

12 RISKS & MITIGATIONS

Risk	Severity	Mitigation
Integration time underestimated	HIGH	Agree on shared data schema Day 1. Reserve Days 13–14 as non-negotiable buffer.
LLM API costs exceed budget	MEDIUM	Use Ollama + CodeLlama-7B locally as free fallback. Test with small code samples.
Fine-tuning fails on free Colab	MEDIUM	Use LoRA (PEFT) — reduces memory by 10x. Verified to run on Colab T4 for CodeBERT/CodeT5.
GitHub webhook flaky during demo	LOW	Keep Streamlit UI as backup demo path. Both routes run the same pipeline code.
Low patch correctness rate	LOW	Even 60–70% accuracy is a valid research finding. Report it honestly with error analysis.

13 DELIVERABLES

Task 1 — April 30, 2026 (50% implementation)

- Working SAST Agent (Bandit + Semgrep + LLM reasoning layer)
- Working Patch Agent with RAG pipeline and self-verification loop

- GitHub webhook → FastAPI endpoint → PR comment working end-to-end
- Fine-tuning code written and at least one baseline model partially trained
- GitHub repository with documented code and README
- Streamlit UI functional for manual demo

Task 2 — ~May 10, 2026 (full project)

- Complete, tested multi-agent pipeline with all three agents
- Both fine-tuned baseline models (CodeBERT + CodeT5) fully trained
- Full evaluation results on SecurityEval benchmark with comparison tables
- Research paper in Springer LNCS format — minimum 8–10 papers cited
- Complete, commented codebase on GitHub
- Demo video or live demo of GitHub PR triggering the pipeline

14 SIGN-OFF

This PRD represents the agreed scope and specification for the AutoSecDev project. Any changes to scope, timeline, or technical approach should be discussed with the course instructor and updated in this document.

Role	Name	Date
Project Owner (Student Team)	_____ —	April 2026
Development Team (Antigravity)	_____ —	_____
Course Instructor	_____ —	_____

— End of Document —